

**Práctica 1 · Mi
primera app
Android: de la
idea al proyecto
con recursos**

Sandro Sanvicente Sánchez



Índice

Práctica 1 · Mi primera app Android: de la idea al proyecto con recursos	1
Parte A · Análisis del desarrollo móvil	2
Ventajas e inconvenientes del desarrollo móvil frente al escritorio/web:	2
Elección Tecnológica	3
Ciclo de vida de una Activity:	4
Parte B · Especificación de tu app	5



Parte A · Análisis del desarrollo móvil

Ventajas e inconvenientes del desarrollo móvil frente al escritorio/web:

El desarrollo móvil ofrece ventajas claras: portabilidad, acceso a hardware integrado y una conexión casi permanente. Sin embargo, tiene inconvenientes notables: las pantallas son mucho más pequeñas, la entrada de datos por teclado táctil es más lenta y estamos sujetos a las políticas de las tiendas de aplicaciones.

Limitaciones concretas aplicadas a la app "CaféNotas":

Batería: Los recursos son limitados. Si la app se quedara en segundo plano buscando actualizaciones constantemente, agotaría la batería. Deberá consumir lo mínimo.

Memoria: Android puede matar procesos si le falta memoria RAM. Si el usuario está a medias de registrar la nota de cata de un café y cambia de app, debemos asegurarnos de guardar el progreso.

Conectividad: Muchas cafeterías tienen mala cobertura al estar en interiores. La app debe permitir guardar los cafés en una base de datos local y sincronizar cuando haya red.

Fragmentación / Tamaño de pantalla: Existen muchos dispositivos Android. La interfaz de registro deberá adaptarse mediante márgenes flexibles para verse bien tanto en un móvil pequeño como en una tablet.

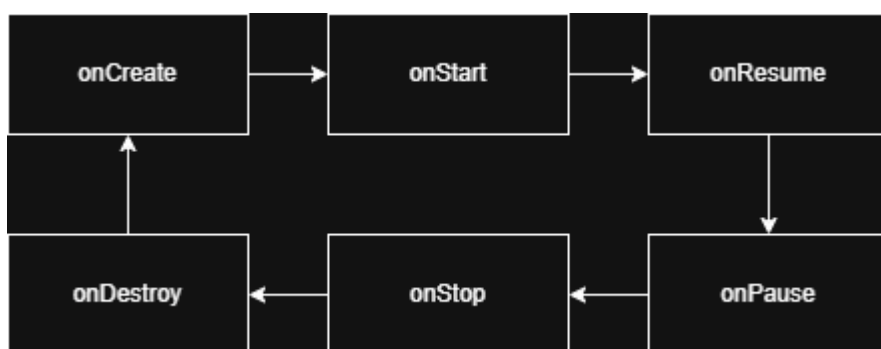


Elección Tecnológica

Enfoque	Lenguajes	Rendimiento	Acceso al Hardware	Coste y Mantenimiento	Ejemplo
Nativo	Java, Kotlin, Swift	Máximo	Total y Nativo	Alto	Android SDK
Híbrido	HTML5, CSS, js	Menor	Limitado	Bajo	Ionic, Apache Cordova
Multiplataforma	Dart, C#	Muy Alto	Alto	Medio	Flutter, React Native

Justificación: La app se desarrollará nativamente en android, porque lo pide el módulo y porque permite un acceso más directo y eficiente al sistema de almacenamiento, asegurando mejor rendimiento.

Ciclo de vida de una Activity:



- **El usuario recibe una llamada:** La app actual pierde el foco temporalmente y pasa a segundo plano. Se ejecuta onPause() y muy probablemente onStop().



- **Gira el móvil:** La actividad actual se destruye y se vuelve a crear para cargar el layout apaisado. El flujo es: onPause() -> onStop() -> onDestroy() y seguidamente onCreate() -> onStart() -> onResume().
- **Pulsa el botón Atrás:** El usuario indica que quiere salir de la pantalla actual. Se ejecuta onPause() -> onStop() -> onDestroy() y la Activity se elimina de la pila.

Parte B · Especificación de tu app

Idea y público objetivo:

CaféNotas es una aplicación móvil diseñada para entusiastas del café de especialidad y baristas aficionados. Permite llevar un registro detallado de las extracciones diarias. El público objetivo son usuarios que buscan estandarizar y mejorar sus recetas de café en casa.

3 Objetivos medibles:

- Permitir al usuario registrar una nueva cata de café en menos de 45 segundos.
- Mostrar el historial de cafés consumidos ordenado por fecha de forma automática al abrir la app.
- Permitir filtrar el listado de cafés por método de extracción.

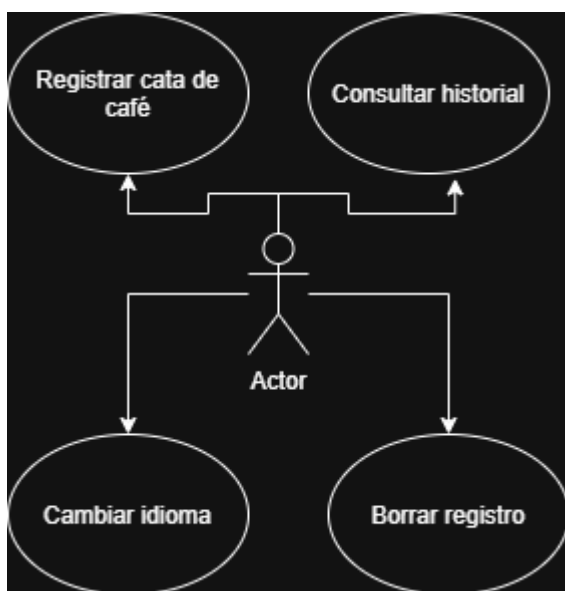
Requisitos funcionales:

- RF-01 (Alta): La app debe permitir registrar un nuevo café guardando origen, método y puntuación.
- RF-02 (Alta): La app debe mostrar una lista con todos los cafés guardados.
- RF-03 (Media): La app debe permitir al usuario borrar un registro de café de la lista.
- RF-04 (Alta): La interfaz de la aplicación debe estar internacionalizada.
- RF-05 (Baja): La app debe permitir aplicar un filtro al listado principal.

Requisitos no funcionales:

- RNF-01 Usabilidad: El botón principal para añadir un nuevo café debe ser accesible con el pulgar derecho en la esquina inferior.
- RNF-02 Compatibilidad: La aplicación debe funcionar en dispositivos con Android 7.0 o superior para abarcar al 95% del mercado.
- RNF-03 Rendimiento: La pantalla principal con el historial debe renderizarse y cargar en menos de 2 segundos.
- RNF-04 Almacenamiento: El tamaño de instalación de la app no debe superar los 15 MB.

Casos de Uso:



Ficha 1: Registrar cata de café

Actor: Usuario.

Precondición: La app está abierta en la pantalla principal.

Flujo principal: El usuario pulsa "+". La app muestra el formulario. El usuario introduce el origen, selecciona el método, da una puntuación y pulsa "Guardar". La app guarda los datos y vuelve al inicio actualizando la lista.



Flujo alternativo: El usuario deja el campo "Origen" vacío y pulsa Guardar. La app muestra un error advirtiéndole que es obligatorio.

Postcondición: El nuevo café queda guardado en el sistema y es visible.

Ficha 2: Cambiar idioma

Actor: Usuario

Precondición: La app está instalada.

Flujo principal: El usuario cambia el idioma general de su teléfono a Inglés desde los ajustes de Android. Al abrir la app, el sistema carga automáticamente los recursos de values-en.

Flujo alternativo: Ninguno.

Postcondición: Toda la interfaz de la app se muestra traducida al inglés.



Parte C. Entorno de desarrollo

El proyecto ha sido creado en Android Studio con el nombre PMDM_P1_SandroSanvicenteSanchez y el paquete personalizado es.medac.sandro.app. Para las pruebas de interfaz y ejecución, se han configurado dos dispositivos virtuales (AVD) con diferentes densidades y tamaños de pantalla:

Dispositivo AVD	Resolución	DPI	Versión API
Pixel7	1080 x 2400	420dpi	API 34
Pixel Tablet	2560 x 1600	320dpi	API 34





Estructura del proyecto:

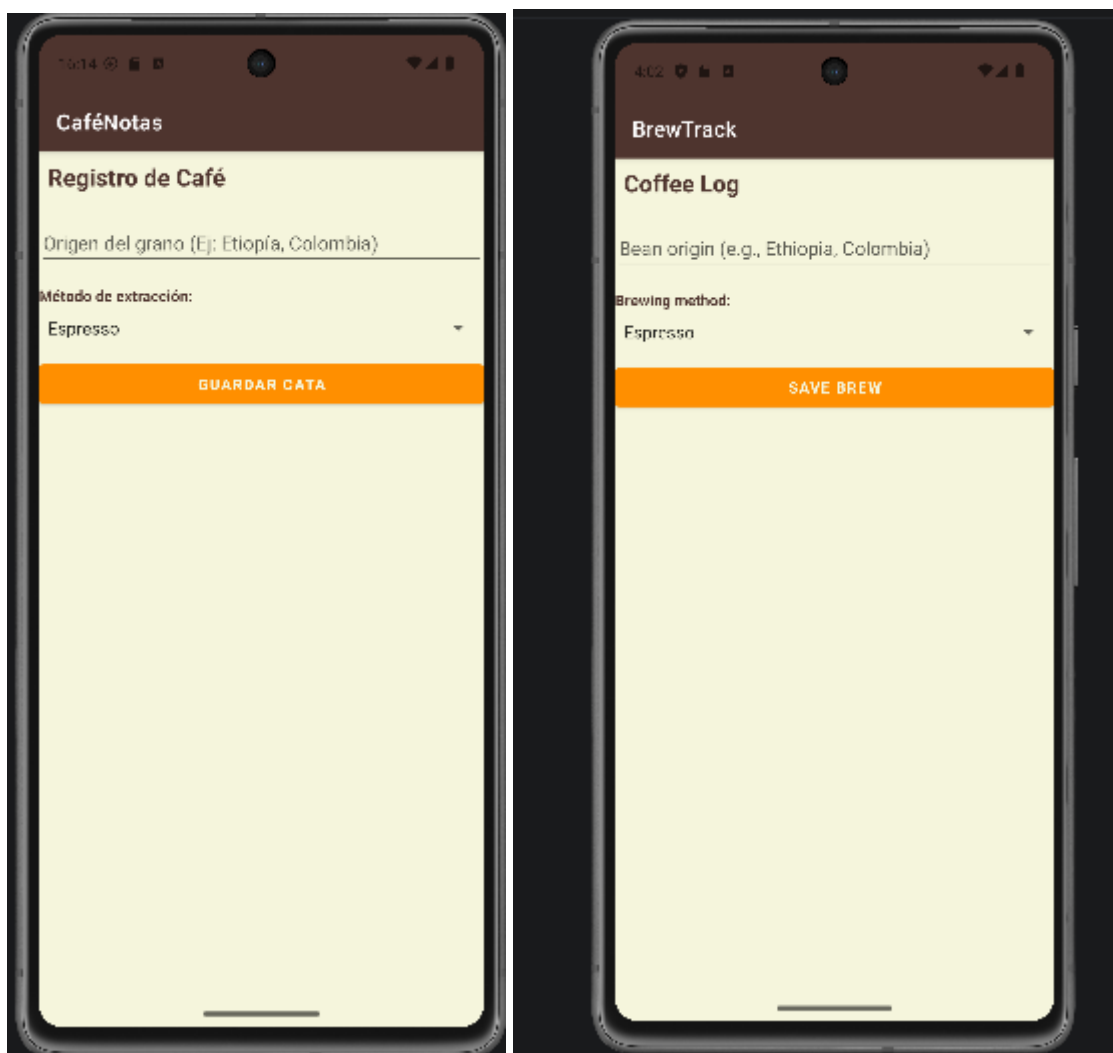
- **AndroidManifest.xml:** Archivo de configuración central. Define el paquete, el icono adaptativo de la app, el tema principal y declara las Activities de la aplicación.
- **Carpeta java/:** Contiene el código fuente organizado por paquetes. Aquí se ubica MainActivity.java, encargada del comportamiento lógico de la interfaz.
- **Carpeta res/:** Almacena los recursos estáticos aislados del código: drawable/ (imágenes vectoriales y mapas de bits), layout/ (diseño XML de las pantallas), mipmap/ (iconos adaptativos del lanzador) y values/ (textos, arrays, paletas de colores y estilos).
- **Ficheros Gradle:** Archivos de configuración de construcción que gestionan las dependencias, la compilación y las versiones de API mínima y objetivo.
- **Relación Activity-Layout:** La clase Java se vincula a su vista gráfica XML mediante la instrucción setContentView(R.layout.activity_main); en el método onCreate()



Parte D. Recursos de la aplicación

Textos e Internacionalización:

Todos los textos visibles se han extraído al archivo strings.xml para evitar cadenas codificadas en duro, incluyendo el uso de un string-array para los métodos de extracción. Se ha implementado la internacionalización añadiendo el calificador de recurso values-en, lo que permite que la aplicación se traduzca automáticamente al inglés si el dispositivo cambia de idioma.





Imágenes e Iconos:

Se ha integrado una imagen vectorial obtenida de Material Symbols porque es infinitamente escalable y no pierde calidad sin importar la densidad de la pantalla, manteniendo el peso del archivo APK al mínimo. También se incluye una imagen de mapa de bits de una taza de café, formato idóneo y necesario para representar fotografías reales que los vectores no pueden procesar eficientemente. El icono principal de la app se ha generado como icono adaptativo en las carpetas mipmap.

Colores, Estilos y Tema:

La aplicación cuenta con una paleta de 5 colores semánticos en colors.xml, justificada por la temática del café de especialidad. Se han creado y reutilizado dos estilos personalizados para estandarizar los títulos y botones, aplicando unidades dp para márgenes/tamaños y sp para las fuentes de texto.

Conclusión personal

Durante la realización de esta práctica, he afianzado los conceptos de separación entre la lógica de negocio y la interfaz gráfica en el entorno de Android Studio. Aprender a centralizar los textos, colores y estilos en la carpeta res/values ha demostrado ser una práctica esencial para mantener un código limpio y facilitar la traducción de la aplicación.

Las principales dificultades encontradas estuvieron relacionadas con la configuración inicial del entorno, específicamente en la activación de la virtualización en la placa base para poder ejecutar el emulador de Android. También hubo retos técnicos menores al configurar el control de versiones con Git desde la propia interfaz de Android Studio y problemas de caché en el buscador interno de iconos vectoriales, los cuales se resolvieron optando por la selección manual y el uso de comandos por terminal para forzar la indexación del repositorio.